

Transformer arxitekturası va matnni umumlashtirish

Tabiiy tilni qayta ishlash — 12-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №12

3-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding
- 4 Transformer bloki va umumlashtirish
- 5 Umumlashtirishni ROUGE bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L11 da attention (*diqqat mexanizmi*) dekoderga manbadagi kerakli so'zlarga “qarash” imkonini berdi. Lekin L11 dagi enkoder-dekoder modeli asosida nima turardi?

2-savol

Manba gapni so'zma-so'z ketma-ket o'qish hisoblash va o'qitishda qanday cheklov tug'diradi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L11 da attention (*diqqat mexanizmi*) dekoderga manbadagi kerakli so'zlarga “qarash” imkonini berdi. Lekin L11 dagi enkoder-dekoder modeli asosida nima turardi?

1-javob

Enkoder ham, dekoder ham **LSTM** edi: h_t ni hisoblash uchun h_{t-1} kerak. Demak, attention bor, ammo asos **rekurrent** qolgan.

2-savol

Manba gapni so'zma-so'z ketma-ket o'qish hisoblash va o'qitishda qanday cheklov tug'diradi?

2-javob

Vaqt bo'yicha **parallellashtirish cheklanadi**: h_t tugamaguncha h_{t+1} hisoblanmaydi. n ta token uchun kamida n ketma-ket qadam kerak. Bugungi yechim — **Transformer**.

Dars oxirida nimalarni bajara olasiz?

- ❶ **Tushuntira olasiz:** Transformer nima sababdan rekurrentlikdan voz kechishini: parallel hisoblash va uzoq masofali bog'lanishlar.
- ❷ **Hisoblay olasiz:** scaled dot-product self-attention (*masshtablangan skalyar ko'paytma attention*) vaznlarini:

$$\alpha = \text{softmax}([2, 1, 3]) = [0.245, 0.090, 0.665].$$

- ❸ **Tushuntira olasiz:** multi-head attention (*ko'p boshli attention*), positional encoding (*pozitsion kodlash*) va residual connection + layer normalization (*qoldiq ulanish + qatlam normalizatsiyasi*) g'oyasini.
- ❹ **Hisoblay olasiz:** umumlashtirish sifatini ROUGE-1 (*reference bilan 1-gram moslik*) orqali:

$$P = 1.0, \quad R = 0.6, \quad F_1 = 0.75.$$

Tayanch bilimlar

L11: attention jarayoni — energiya $\rightarrow$ softmax $\rightarrow$ kontekst. L6/L9: softmax. Matematikadan: skalyar ko'paytma, matritsa ko'paytma, exp, sin / cos.

Bugungi dars rejası

Bosqich	Mavzu	Amaliy natija
1	Self-attention va parallellashtirish	$\alpha = [0.245, 0.090, 0.665]$
2	Multi-head attention va positional encoding	$PE(0) = [0, 1, \dots]$
3	Transformer bloki va umumlashtirish	residual connection + Layer-Norm
4	ROUGE-1 bilan baholash	$F_1 = 0.75$
	Xulosa va 12-amaliyotga ko'rsatmalar	Amaliyotda quriladigan model

L11 dan L12 ga ko'prik

L11 da attention dekoderning manbaga qarash mexanizmi edi. Bugun shu g'oya self-attention orqali Transformer blokiga olib boriladi. Oxirida umumlashtirish natijasi ROUGE-1 bilan baholanadi.

Bugungi muammo: rekurrent hisob parallellashtirishni cheklaydi

RNN/LSTM qayerda qiynaladi?

Ketma-ket hisob

h_t ni hisoblash uchun h_{t-1} kerak. Shuning uchun n ta token uchun hisob kamida n ketma-ket qadamdan o'tadi — vaqt bo'yicha **parallellashtirish cheklanadi**.

Uzoq bog'lanish sust

Uzoq joylashgan ikki token orasidagi axborot ko'p oraliq holatlardan o'tadi. Natijada gradient yo'lda zaiflashishi mumkin (*vanishing gradient*).

Bugungi yechim yo'nalishi

- 1 Rekurrent bog'lanishni asosiy mexanizm sifatida olib tashlaymiz.
- 2 Har bir token boshqa tokenlar bilan **bir vaqtda** bog'lanadi: self-attention.
- 3 Bir attention qatlamida ikki pozitsiya orasidagi yo'l **to'g'ridan-to'g'ri** bo'ladi.

Attention Is All You Need g'oyasi: rekurrent qatlamlar shart emas; attention Transformerda asosiy kontekst yig'ish mexanizmi bo'la oladi.

Izoh: self-attention pozitsiyalar bo'yicha parallel, lekin uzunlik n bo'lsa, attention matritsasi odatda $O(n^2)$ hisob talab qiladi.

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding
- 4 Transformer bloki va umumlashtirish
- 5 Umumlashtirishni ROUGE bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Cross-attention va self-attention: aniq farq

Jihat	Cross-attention (L11)	Self-attention (bugun)
Q qayerdan	dekoder holatidan	shu ketma-ketlikning o'zidan
K, V qayerdan	enkoder chiqishidan, ya'ni manba tokenlardan	shu ketma-ketlikning o'zidan
Kim kimga qaraydi?	dekoder tokeni $\rightarrow$ manba tokenlar	har token $\rightarrow$ shu ketma-ketlikdagi tokenlar
Asosiy vazifa	tarjimada manba-target mosligini topish	har token uchun kontekstli ifoda qurish

L11 dan ko'prik

Asosiy hisoblash naqshi bir xil:

score $\rightarrow$ softmax $\rightarrow$ vaznlangan yig'indi.

Farq shunda: cross-attentionda Q dekoderdan, K, V enkoderdan keladi; self-attentionda esa Q, K, V bitta ketma-ketlikdan olinadi.

Intuitsiya: har token gap ichidan kontekst yig'adi

L11 dan ko'prik. L11 da **dekoder** enkoder chiqishlariga qarardi (*cross-attention*). Self-attention esa shu g'oyani **bitta ketma-ketlik ichida** qo'llaydi.

Har token shu gapdagi tokenlardan, odatda o'zidan ham, axborot yig'adi. Natijada tokenning ifodasi **kontekstga mos** bo'lib qoladi.

Mini-misol

Olma qizil. $\Rightarrow olma = meva$

Kitobni **olma**. $\Rightarrow olma = buyruqning inkori$

Atrofdagi so'zlar bir xil yozilgan tokenning ma'nosini aniqlashga yordam beradi.

Asosiy g'oya

Self-attention (*o'z-o'ziga attention*)da so'rov (*query*) endi dekoderdan emas, shu ketma-ketlikning o'zidan keladi.

moslik balli $\rightarrow$ softmax $\rightarrow$ kontekst

Hisoblash naqshi L11 attention bilan bir xil, farq Q, K, V qayerdan olinishi bilan bog'liq.

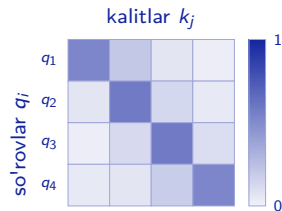
Scaled dot-product attention: rasmiy ta'rif

Ta'rif: $\text{Attention}(Q, K, V)$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

bunda: Q — so'rovlar (query); K — kalitlar (key); V — qiymatlar (value); d_k — kalit o'lchami; $QK^\top / \sqrt{d_k}$ — masshtablangan moslik energiyasi; softmax **qatorlar** bo'yicha.

Self-attention'da $Q = XW^Q$, $K = XW^K$, $V = XW^V$ — uchchalasi **bitta** kirish X dan. L11 attention'ning matritsali, parallel shakli.



$\alpha = \text{softmax}(QK^\top / \sqrt{d_k})$: har **qator** kalitlarga vazn beradi (yig'indi = 1), so'ng $\times V$.

Nega attention'da $\sqrt{d_k}$ ga bo'lamiz?

Qisqa keltirib chiqarish

1-qadam. Moslik balli so'rov va kalitning skalyar ko'paytmasi:

$$e_{ij} = q_i^\top k_j.$$

2-qadam. Agar komponentlar odatiy masshtabda bo'lsa, d_k kattalashganda bu yig'indining dispersiyasi ham taxminan d_k bilan ortadi.

3-qadam. Juda katta yoki juda kichik energiyalar softmax'ni **to'yintiradi**: ehtimol bir pozitsiyaga yaqinlashadi, gradient esa kichrayadi.

4-qadam. Shuning uchun

$$\frac{q_i^\top k_j}{\sqrt{d_k}}$$

ishlatiladi: ballar masshtabi me'yorlashadi va softmax barqarorroq bo'ladi.



Intuitsiya

Masshtabsiz: softmax bitta ustunga yopishib qolishi mumkin.

Masshtablangan: taqsimot yumshoqroq bo'ladi va o'qitish barqarorlashadi.

Qo'lda misol: self-attention vaznlari

So'rov–kalit mosliklari: soddalashtirilgan misol, $d_k = 4$

$$q \cdot k_1 = 4, \quad q \cdot k_2 = 2, \quad q \cdot k_3 = 6.$$

$$e = \frac{[4, 2, 6]}{\sqrt{4}} = \frac{[4, 2, 6]}{2} = [2, 1, 3].$$

Softmax hisob:

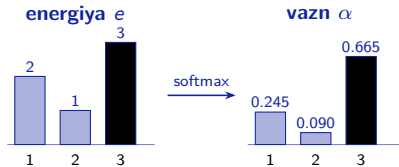
$$Z = e^2 + e^1 + e^3$$

$$\approx 7.389 + 2.718 + 20.086 = 30.193,$$

$$\alpha = \frac{[7.389, 2.718, 20.086]}{30.193}$$

$$\approx [0.245, 0.090, 0.665]$$

Demak, 3-pozitsiya eng katta energiyaga ega $\Rightarrow$ eng katta attention vaznini oladi.



Softmax energiyalarni yig'indisi 1 bo'lgan vaznlarga aylantiradi.

Kontekst vektori

Agar $v_1 = [1, 0]$, $v_2 = [0, 1]$, $v_3 = [1, 1]$ bo'lsa, kontekst $\sum_i \alpha_i v_i = 0.245v_1 + 0.090v_2 + 0.665v_3 = [0.910, 0.755]$ — token yakuniy ifodani qiymat vektorlarining vaznlangan yig'indisidan oladi.

Mashq: boshqa so'rov uchun attention vaznlari

Savol

Berilgan:

$$q \cdot k = [2, 2, 4], \quad d_k = 4 \quad (\sqrt{d_k} = 2).$$

Shuning uchun masshtablangan energiyalar:

$$e = \frac{[2, 2, 4]}{\sqrt{4}} = [1, 1, 2].$$

$$\alpha = \text{softmax}([1, 1, 2]) = ?$$

$$(e^1 \approx 2.718, e^2 \approx 7.389)$$

Ko'rsatma

Har bir energiyani eksponentlang, yig'indisi Z ga bo'ling: $\alpha_i = e^{x_i} / Z$. Teng energiyalar teng vazn oladi.

Mashq: boshqa so'rov uchun attention vaznlari

Javob

$$Z = e^1 + e^1 + e^2$$

$$\approx 2.718 + 2.718 + 7.389 = 12.825,$$

$$\alpha = \frac{[2.718, 2.718, 7.389]}{12.825} \approx [0.212, 0.212, 0.576].$$

Teng energiyali 1- va 2-pozitsiyalar teng vazn oladi; 3-pozitsiya eng katta energiyaga ega, shu bois vazni ham eng katta. Vaznlar yig'indisi 1 ga teng.



3-pozitsiya → eng katta vazn

Kod ↔ formula: scaled dot-product attention

Soddalashtirilgan NumPy misoli:

```
import numpy as np
def self_attention(X, Wq, Wk, Wv):
    Q = X @ Wq
    K = X @ Wk
    V = X @ Wv
    d_k = K.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    scores = scores - scores.max(axis=-1, keepdims=True)
    alpha = np.exp(scores)
    alpha = alpha / alpha.sum(axis=-1, keepdims=True)
    return alpha @ V
```

Kod → formula

Kod	Formula
Q @ K.T	QK^T
/ sqrt(d_k)	$/\sqrt{d_k}$
exp / sum	softmax
alpha @ V	$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

Muhim izoh

scores - scores.max — softmaxni sonli barqaror qiladi. Kod barcha tokenlar uchun vaznlarni bitta matritsa amalda hisoblaydi.

Amaliy Transformerda padding yoki causal mask ham qo'shilishi mumkin; bu slaydda asosiy formula ko'rsatilgan

Self-attention tuzog'i: masshtabsiz softmax to'lyinadi

$\sqrt{d_k}$ ni unutish

Agar $q^\top k$ ballari $\sqrt{d_k}$ ga bo'linmasa, d_k katta bo'lganda energiyalar haddan tashqari kattalashishi mumkin.

Natijada softmax juda keskinlashadi: bitta pozitsiya ≈ 1 , qolganlari ≈ 0 vazn oladi. Bu ko'p pozitsiyalar uchun gradientni **juda kichraytiradi**.

Yechim

Attention ballarini masshtablaymiz: $q^\top k \frac{1}{\sqrt{d_k}}$
Bu softmaxni yumshoqroq qiladi va o'qitishni barqarorlashtiradi.

Eslatma

Self-attention vaqt bo'yicha parallel hisoblanadi, lekin n ta token uchun attention matritsasi odatda $O(n^2)$ hisob va xotira talab qiladi.

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding**
- 4 Transformer bloki va umumlashtirish
- 5 Umumlashtirishni ROUGE bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: bir nechta “qarash” va tartib signali

Multi-head attention

Bitta head (*attention boshi*) — bitta attention hisobidir.

Bir nechta head parallel ishlaydi va tokenlar orasidagi turli bog'lanishlarni o'rganishi mumkin: masalan, biri sintaktik, boshqasi semantik aloqaga sezgir bo'lishi mumkin.

Positional encoding

Self-attentionning o'zi tokenlarning tartibini alohida bilmaydi. Shuning uchun har pozitsiyaga maxsus tartib signali qo'shiladi:

$$\tilde{x}_{pos} = x_{pos} + PE_{pos}.$$

Masalan, “men seni” va “seni men” bir xil so'zlardan tuzilgan, lekin tartibi boshqa.

Asosiy xulosa

Multi-head attention bir nechta “qarash nuqtasi” beradi; positional encoding esa token vektoriga tartib haqidagi signalni qo'shadi. Oldingi qo'lda hisoblangan α misoli — **bitta head** uchun edi.

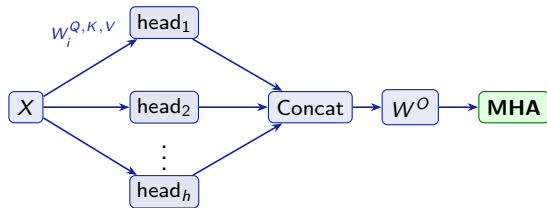
Multi-head attention va positional encoding: rasmiy ta'rif

Ta'rif: multi-head attention va PE

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad \text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Belgilashlar: h — head soni; $W_i^{Q,K,V}$ — i -head uchun proyeksiya matritsalar; W^O — chiqish proyeksiyasi; pos — pozitsiya; i — o'lcham indeksi; d — model o'lchami.



Juft indekslarda sin, toq indekslarda cos. Turli o'lchamlar turli chastotali **pozitsion signal** beradi.

Har bir head alohida attention hisoblaydi; natijalar birlashtirilib W^O orqali chiqishga o'tkaziladi.

Nega multi-head va positional encoding kerak?

Multi-head nima uchun?

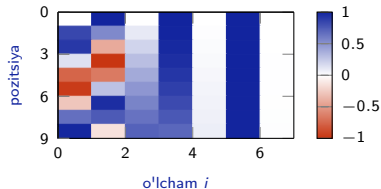
- 1 Bitta head bitta proyeksiya fazosida attention hisoblaydi.
- 2 h ta head kirishni turli kichik fazolarga proyeksiya qiladi.
- 3 Model turli bog'lanishlarni parallel o'rganadi.

Positional encoding nima uchun?

- 1 PE bo'lmasa, self-attention tartib signalini bilmaydi.
- 2 Tokenlar almasha, tartib haqidagi axborot yo'qoladi.
- 3 Sinusoidal PE har pozitsiyaga farqlanuvchi signal beradi.

Xulosa

Multi-head — bir nechta attention “qarashi”; PE — token vektoriga qo'shiladigan tartib signali.



PE heatmap intuitsiyasi

Har bir qator — bitta pozitsiya. Har bir ustun — PE vektorining bir o'lchami. Turli chastotalar pozitsiyalarni bir-biridan farqlashga yordam beradi.

Qo'lda misol: 0-pozitsiya uchun positional encoding

Positional encoding: $pos = 0, d = 4$

Indekslash 0 dan boshlansin. Shunda birinchi token uchun $pos = 0$ bo'ladi.

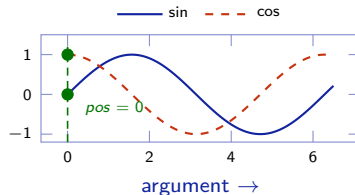
$$PE_{(0,0)} = \sin(0), \quad PE_{(0,1)} = \cos(0), \quad PE_{(0,2)} = \sin(0), \quad PE_{(0,3)} = \cos(0).$$

$$\text{Chunki } pos = 0 \Rightarrow \frac{pos}{10000^{2i/d}} = 0 \quad \text{barcha } i \text{ lar uchun.}$$

$$\sin(0) = 0, \quad \cos(0) = 1 \Rightarrow PE(0) = [0, 1, 0, 1].$$

$$PE(0) = \begin{matrix} & \sin & \cos & \sin & \cos \\ \begin{matrix} \boxed{0} & \boxed{1} & \boxed{0} & \boxed{1} \end{matrix} \\ i=0 & i=1 & i=2 & i=3 \end{matrix}$$

Juft indeks — sin, toq indeks — cos.



Asosiy g'oya

Pozitsiya signali token embeddingiga qo'shiladi: $\tilde{x}_0 = x_0 + PE(0)$, ya'ni bu misolda qo'shiladigan vektor $PE(0) = [0, 1, 0, 1]$.

Mashq: $pos = 1$ uchun to'liq PE vektori ($d = 4$)

Savol

Indekslash 0 dan: ikkinchi token uchun $pos = 1$. Sinusoidal PE:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

$d = 4$ uchun ikkala juftni ($i = 0, 1$) hisoblab, to'liq vektorni toping:

$$PE(1) = [PE_{(1,0)}, PE_{(1,1)}, PE_{(1,2)}, PE_{(1,3)}] = ?$$

Berilgan: $\sin 1 \approx 0.841$, $\cos 1 \approx 0.540$; kichik x uchun $\sin x \approx x$, $\cos x \approx 1$.

Mashq: $pos = 1$ uchun to'liq PE vektori ($d = 4$)

Savol

Indekslash 0 dan: ikkinchi token uchun $pos = 1$. Sinusoidal PE:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right).$$

$d = 4$ uchun ikkala juftni ($i = 0, 1$) hisoblab, to'liq vektorni toping:

$$PE(1) = [PE_{(1,0)}, PE_{(1,1)}, PE_{(1,2)}, PE_{(1,3)}] = ?$$

Berilgan: $\sin 1 \approx 0.841$, $\cos 1 \approx 0.540$; kichik x uchun $\sin x \approx x$, $\cos x \approx 1$.

Javob

Har juft $[\sin(\arg), \cos(\arg)]$, $\arg = pos/10000^{2i/d}$:

$$i = 0 : 10000^0 = 1, \arg = 1 \Rightarrow [0.841, 0.540],$$

$$i = 1 : 10000^{1/2} = 100, \arg = 0.01 \Rightarrow [0.010, 1.000].$$

$$PE(1) \approx [0.841, 0.540, 0.010, 1.000].$$

Turli chastotalar sabab har pozitsiya o'ziga xos signal oladi.

	sin	cos	sin	cos
$pos = 0$	0	1	0	1
$pos = 1$	0.841	0.540	0.010	1.000
	$i = 0$	$i = 1$	$i = 2$	$i = 3$

$PE(0)$ va $PE(1)$ farqli: model tartibni shu farqdan oladi.

Kod $\leftrightarrow$ formula: positional encoding

Sinusoidal PE uchun NumPy kodi:

```
import numpy as np
def positional_encoding(n_pos, d):
    pos = np.arange(n_pos)[: , None] # (n_pos, 1)
    j = np.arange(d)[None, :] # (1, d)
    div = 10000 ** (2 * (j // 2) / d)
    angle = pos / div
    pe = np.zeros((n_pos, d))
    pe[:, 0::2] = np.sin(angle[:, 0::2])
    pe[:, 1::2] = np.cos(angle[:, 1::2])
    return pe
pe = positional_encoding(2, 4)
print(pe[0])
print(np.round(pe[1], 3))
```

Kod $\rightarrow$ formula

Kod	Ma'nosi
$j // 2$	juftlik indeksi i
angle	$\frac{pos}{10000^{2i/d}}$
$0::2$	$PE_{(pos, 2i)} = \sin(\cdot)$
$1::2$	$PE_{(pos, 2i+1)} = \cos(\cdot)$

Natija

Kod quyidagini beradi:

$$PE(0) = [0, 1, 0, 1],$$

$$PE(1) \approx [0.841, 0.540, 0.010, 1.000].$$

Izoh

PE token embeddingiga qo'shiladi ($\tilde{x} = x + PE$) va formula bilan beriladi — o'rganiladigan parametr emas.

PE tuzog'i: positional encoding bo'lmasa, tartib signali yo'lq

Tartib signalisiz model

Self-attentionning o'zi tokenlarning tartibini alohida kodlamaydi. Agar positional encoding qo'shilmasa, model asosan tokenlar orasidagi moslikka qaraydi, lekin **qaysi token qayerda turgani** noaniq qoladi.

Masalan:

“faqat Ali Valiyni ko'rdi” $\neq$ “Ali faqat Valiyni ko'rdi”.

Bir xil so'zlar boshqa tartibda kelsa, urg'u va ma'no o'zgarishi mumkin.

Yechim

Token embeddingiga pozitsiya signali qo'shiladi:

$$\tilde{x}_{pos} = x_{pos} + PE_{pos}.$$

Shunda model tokenning **mazmuni** bilan birga uning **gapdagi o'rni** haqida ham signal oladi.

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding
- 4 Transformer bloki va umumlashtirish**
- 5 Umumlashtirishni ROUGE bilan baholash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: Transformer bloklari ustma-ust joylanadi

Bitta Transformer bloki

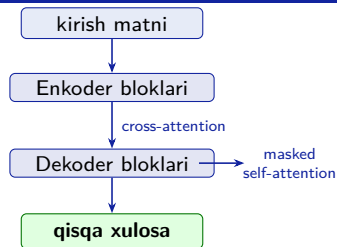
Har blokda ikki asosiy qism bor:

- 1 **Multi-head self-attention**: tokenlar bir-biri bilan bog'lanadi.
- 2 **FFN** (*feed-forward network*): har token alohida qayta ishlanadi.

Har qism atrofida odatda residual connection + LayerNorm ishlatiladi.

Asosiy g'oya

Bloklar N marta ustma-ust qo'yiladi. Pastki qatlamlar sodda bog'lanishlarni, yuqori qatlamlar esa murakkabroq kontekstni ifodalaydi.



Abstrakt umumlashtirish jarayoni

Enkoder kirish matnini o'qib, kontekstli ifodalar hosil qiladi.

Dekoder qisqa xulosani tokenma-token yaratadi: masked self-attention oldingi xulosa tokenlariga qaraydi, cross-attention esa enkoder chiqishidan foydalanadi.

Transformer bloki: sublayer, residual va LayerNorm

Rasmiy ta'rif

Har bir sublayer uchun:

$$y = \text{Sublayer}(x), z = \text{LayerNorm}(x+y).$$

FFN sublayer:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2.$$

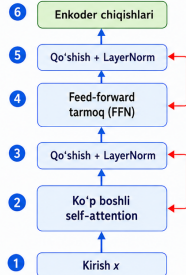
Bu yerda x — sublayer kirishi, y — sublayer natijasi, z — normallashtirilgan keyingi chiqish.

Qanday o'qiladi?

- **Attention** tokenlar orasidan kontekst yig'adi.
- **FFN** har tokenni alohida qayta ishlaydi.
- **Residual + LayerNorm** signalni saqlaydi va o'qitishni barqarorlashtiradi.

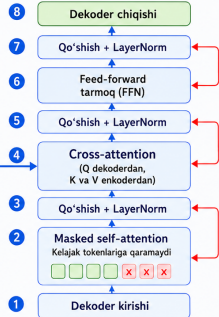


Enkoder bloki



K, V: enkoder chiqishlari

Dekoder bloki



Transformer arxitekturasining pedagogik soddalashtirilgan ko'rinishi.

Enkoderda: self-attention → FFN.

Dekoderda: masked self-attention → cross-attention → FFN.

Nega residual, LayerNorm, FFN va mask kerak?

Asosiy sabablar

- ① **Residual:** chuqur bloklarda gradient zaiflashishi mumkin.

$$x + \text{Sublayer}(x)$$

signal va gradient uchun **qisqa yo'l** beradi.

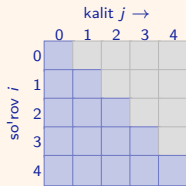
- ② **LayerNorm:** aktivatsiyalar masshtabi qatlamlar bo'ylab o'zgarishi mumkin. LayerNorm har token vektorini normallab, o'qitishni **barqarorlashtiradi**.

- ③ **FFN:** attention kontekst yig'adi, FFN esa har tokenni alohida **nochiziqli** qayta ishlaydi.

- ④ **Causal mask:** dekoder i -pozitsiyada turib, faqat $j \leq i$ tokenlarga qarashi kerak.

$$j > i \Rightarrow e_{ij} = -\infty \Rightarrow \alpha_{ij} = 0$$

Causal mask intuitsiyasi



Ko'k: ko'radi

Kulrang: maskalanadi

Xulosa

Residual + **LayerNorm** — barqaror chuqurlik,
FFN — ifodalash quvvati,
mask — to'g'ri avtoregressiv generatsiya.

Qo'lda misol: residual va LayerNorm

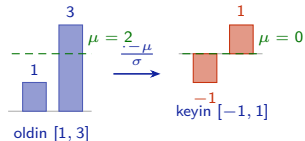
1-qadam: residualdan keyingi vektor

$$x = [0, 1], \quad \text{Sublayer}(x) = [1, 2]$$
$$y = x + \text{Sublayer}(x) = [1, 3].$$

2-qadam: LayerNorm (soddalashtirilgan)

Bu yerda ε hamda affine parametrlar (γ, β) e'tiborsiz:

$$\mu = \frac{1+3}{2} = 2$$
$$\sigma^2 = \frac{(1-2)^2 + (3-2)^2}{2} = 1, \quad \sigma = 1$$
$$\hat{y} = \frac{y - \mu}{\sigma} = \frac{[1, 3] - 2}{1} = [-1, 1].$$



O'rtacha $2 \rightarrow 0$ ga suriladi, masshtab normallanadi.

Mashq: uch o'lchamli vektorni LayerNorm qiling

Savol

Vektor berilgan: $\mathbf{v} = [1, 2, 3]$.

ε , γ va β ni e'tiborsiz qoldirib, quyidagi uch qadamni bajaring:

$$\mu = \frac{v_1 + v_2 + v_3}{3}, \sigma = \sqrt{\frac{(v_1 - \mu)^2 + (v_2 - \mu)^2 + (v_3 - \mu)^2}{3}}, \hat{\mathbf{v}} = \frac{\mathbf{v} - \mu}{\sigma}.$$

Toping: (μ) , (σ) va normallashtirilgan vektor $(\hat{\mathbf{v}})$.

Mashq: uch o'lchamli vektorni LayerNorm qiling

Savol

Vektor berilgan: $\mathbf{v} = [1, 2, 3]$.

ε , γ va β ni e'tiborsiz qoldirib, quyidagi uch qadamni bajaring:

$$\mu = \frac{v_1 + v_2 + v_3}{3}, \sigma = \sqrt{\frac{(v_1 - \mu)^2 + (v_2 - \mu)^2 + (v_3 - \mu)^2}{3}}, \hat{\mathbf{v}} = \frac{\mathbf{v} - \mu}{\sigma}.$$

Toping: (μ) , (σ) va normallashtirilgan vektor $(\hat{\mathbf{v}})$.

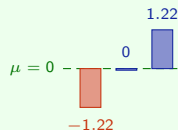
Javob

$$[\mu = \frac{1+2+3}{3} = 2.]$$

$$\sigma = \sqrt{\frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3}} = \sqrt{\frac{2}{3}} \approx 0.816.$$

$$\hat{\mathbf{v}} = \frac{[1, 2, 3] - 2}{0.816} = \frac{[-1, 0, 1]}{0.816} \approx [-1.22, 0, 1.22].$$

Normallashtirish vektorni markazlaydi: o'rtacha qiymat nolga aylanadi.



Natija nol atrofida joylashadi.

Kod $\leftrightarrow$ formula: Transformer enkoder bloki

Post-LN encoder block:

```
import torch.nn as nn

class EncoderBlock(nn.Module):
    def __init__(self, d, h, ff):
        super().__init__()
        self.mha = nn.MultiheadAttention(
            d, h, batch_first=True)
        self.ffn = nn.Sequential(
            nn.Linear(d, ff),
            nn.ReLU(),
            nn.Linear(ff, d))
        self.n1 = nn.LayerNorm(d)
        self.n2 = nn.LayerNorm(d)

    def forward(self, x):
        x0 = x
        a, _ = self.mha(
            x0, x0, x0,
            need_weights=False)
        x1 = self.n1(x0 + a)
        f = self.ffn(x1)
        z = self.n2(x1 + f)
        return z
```

Kod $\rightarrow$ formula

Kod	Formula
$x_0 = x$	$x_0 = x$
$mha(x_0, x_0, x_0)$	$a = MHA(x_0, x_0, x_0)$
$n1(x_0 + a)$	$x_1 = LN(x_0 + a)$
$ffn(x_1)$	$f = FFN(x_1)$
$n2(x_1 + f)$	$z = LN(x_1 + f)$

Asosiy qoida

Enkoder blokida ikki sublayer bor:

SelfAttention $\rightarrow$ FFN.

Har bir sublayerdan keyin:

kiritma + sublayer(kiritma) $\rightarrow$ LayerNorm.

Ya'ni bu **Post-LN residual** ko'rinishidir.

Transformer tuzog'i: dekodeerda causal mask shart

Maskni unutish

O'qitishda dekodeerga target ketma-ketligi beriladi. Agar **causal mask** qo'yilmasa, i -pozitsiyadagi token keyingi tokenlarga ham qarab qo'yadi.

Natija: training loss past ko'rinishi mumkin, lekin generatsiyada model kelajak tokenlarini ko'ra olmaydi. Bu **train–inference mismatch** keltiradi.

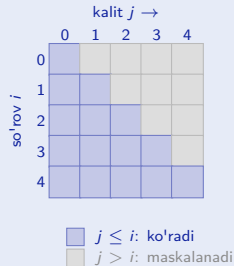
Yechim

Dekoder self-attentionida kelajak pozitsiyalar yopiladi:

$$j > i \Rightarrow e_{ij} = -\infty \Rightarrow \alpha_{ij} = 0.$$

Demak, i -pozitsiya faqat $j \leq i$ tokenlarni ko'radi.

Causal mask intuitsiyasi



Causal mask faqat dekodeerda self-attention qismida ishlatiladi. Cross-attentionida dekodeer enkoder chiqishlariga qaraydi.

Transformer'ni qo'llash: matnni umumlashtirish

Ikki yondashuv

Ekstraktiv umumlashtirish (*extractive summarization*): matndan tayyor gap yoki iboralarni tanlab oladi.

Abstraktiv umumlashtirish (*abstractive summarization*): manba mazmunini qisqa, yangi ifoda bilan qayta yozadi.

Diqqat: abstraktiv model manbada yo'q faktni qo'shib yuborishi mumkin (*hallucination*) — faktga sodiqlik tekshiriladi.

Transformer qanday ishlaydi?

Encoder-decoder Transformerda:

- **enkoder** kirish matnini kontekstli ifodalarga aylantiradi;
- **dekoder** qisqa xulosani tokenma-token generatsiya qiladi.

O'zbekcha misol: yangilik → xulosa

Kirish matni:

“Toshkentda bugun yangi metro bekati ochildi. Bekatdan kun davomida minglab yo'lovchilar foydalandi.”

Xulosa:

“Toshkentda yangi metro bekati ochildi.”

Keyingi savol

Bu xulosa qanchalik yaxshi?

Keyingi bo'limda avtomatik baholash uchun **ROUGE** metrikasini ko'ramiz.

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding
- 4 Transformer bloki va umumlashtirish
- 5 Umumlashtirishni ROUGE bilan baholash**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: xulosa reference birliklarni qanchalik qamradi?

Asosiy savol

Umumlashtirishda model xulosasi **reference** yoki **reference xulosa** bilan solishtiriladi.

ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) xulosa va reference orasidagi **leksik ustma-ustlikni** o'lchaydi: model referencedagi muhim so'z yoki iboralarni qanchalik qamradi?

Muhim cheklov

ROUGE ma'no va fakt sodiqligini to'liq tekshirmaydi. Parafraza yaxshi bo'lsa ham, so'zlar mos kelmasa ROUGE past chiqishi mumkin. Shuning uchun ROUGE — **taxminiy avtomatik baho**.

ROUGE turlari

ROUGE-1: unigram, ya'ni bitta so'z mosligi.

ROUGE-2: bigram, ya'ni ketma-ket kelgan ikki so'z mosligi.

ROUGE-L: eng uzun umumiy ketma-ketlik (*Longest Common Subsequence*, LCS).

Har biri uchun odatda:

Precision, Recall, F_1

hisoblanadi.

BLEU bilan farq

BLEU tarjimada ko'proq aniqlik va qisqalik jarimasiga tayanadi. ROUGE esa umumlashtirishda **qamrovni**, ya'ni recallni ham muhim ko'radi.

ROUGE-1: rasmiy ta'rif

Ta'rif: ROUGE-1 (unigram)

$$M_1 = \sum_w \min(c_{\text{hyp}}(w), c_{\text{ref}}(w))$$
$$P = \frac{M_1}{|\text{hyp}|}, \quad R = \frac{M_1}{|\text{ref}|}, \quad F_1 = \frac{2PR}{P + R}$$

Belgilar ma'nosi

- $c_{\text{hyp}}(w)$ — generatsiyadagi w unigram soni
- $c_{\text{ref}}(w)$ — referencedagi w unigram soni
- M_1 — **clipped** mos unigramlar soni
- $|\text{hyp}|, |\text{ref}|$ — jami unigramlar soni

Eslatma

BLEU ko'proq **precision**ga tayanadi; ROUGE esa **recall**ni ham muhim ko'radi. Shu sabab u umumlashtirishda ko'p ishlatiladi.

Kichik misol

Reference: A B B C

Hypothesis: A B D

Mos unigramlar:

$$M_1 = \min(1, 1) + \min(1, 2) = 2$$

$$P = \frac{2}{3} \approx 0.67, \quad R = \frac{2}{4} = 0.50$$

$$F_1 = \frac{2PR}{P + R} \approx 0.57$$

Nega precision, recall va F_1 kerak?

ROUGE-1 uchun uchta asosiy o'lchov

Recall — reference xulosadagi unigramlarning qancha qismi model xulosasida topildi?

$$R = \frac{\text{mos unigramlar}}{\text{reference unigramlari}}$$

Bu muhim birliklar tushib qolmaganini ko'rsatadi.

Precision — model xulosasidagi unigramlarning qancha qismi referenceda bor?

$$P = \frac{\text{mos unigramlar}}{\text{hypothesis unigramlari}}$$

Bu ortiqcha yoki keraksiz birliklar ko'paymaganini nazorat qiladi.

F_1 — precision va recallning garmonik o'rtachasi:

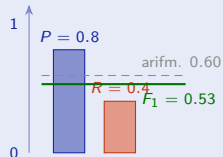
$$F_1 = \frac{2PR}{P + R}$$

Agar P va R orasida nomuvozanat bo'lsa, F_1 pastroq chiqadi.

Qisqa xulosa

Recall — **qamrov**, precision — **aniqlik**, F_1 esa ikkalasi orasidagi **muvozanat**ni beradi.

Kichik misol



$$P = 0.8, \quad R = 0.4$$
$$F_1 = \frac{2PR}{P + R} = \frac{2 \cdot 0.8 \cdot 0.4}{0.8 + 0.4} \approx 0.53.$$

F_1 arifmetik o'rtachadan pastroq, chunki u kichik qiymat tomonga tortiladi.

Eslatma

ROUGE asosan leksik moslikni o'lchaydi; fakt sodiqligi va parafrazani har doim to'liq baholamaydi.

Qo'lda hisob: ROUGE-1 ($F_1 = 0.75$)

reference va model xulosasi

Reference: nlp juda qiziq va foydali

$|ref| = 5$

Hypothesis (model): nlp juda foydali

$|hyp| = 3$

Hisoblash

Mos (**clipped**) unigramlar:

nlp, juda, foydali

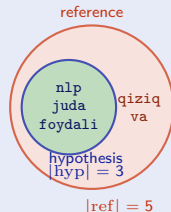
$$M_1 = 3$$

$$P = \frac{M_1}{|hyp|} = \frac{3}{3} = \mathbf{1.00}$$

$$R = \frac{M_1}{|ref|} = \frac{3}{5} = \mathbf{0.60}$$

$$F_1 = \frac{2PR}{P + R} = \frac{2 \cdot 1.0 \cdot 0.6}{1.0 + 0.6} = \mathbf{0.75}.$$

Vizual intuitsiya



- Modeldagi barcha unigramlar referenceda bor $\Rightarrow P = 1.0$.
- referencedagi qiziq va va tushib qolgan $\Rightarrow R = 0.6$.
- Shuning uchun yakuniy muvozanat bahosi $F_1 = 0.75$.

Bu rasm faqat intuitsiya uchun; ushbu misolda takror yo'q, shuning uchun clipped overlap oddiy moslik bilan bir xil.

Mashq: boshqa hypothesis uchun ROUGE-1

Savol

Reference: nlp juda qiziq va foydali

$|ref| = 5$

Hypothesis (model): nlp foydali yaxshi

$|hyp| = 3$

Mos unigramlarni toping va ROUGE-1 ni hisoblang:

$P = ?$, $R = ?$, $F_1 = ?$.

Mashq: boshqa hypothesis uchun ROUGE-1

Savol

Reference: nlp juda qiziq va foydali

$|ref| = 5$

Hypothesis (model): nlp foydali yaxshi

$|hyp| = 3$

Mos unigramlarni toping va ROUGE-1 ni hisoblang:

$P = ?$, $R = ?$, $F_1 = ?$.

Javob

Mos unigramlar:

nlp, foydali $\Rightarrow M_1 = 2$.

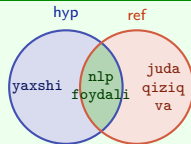
$$P = \frac{M_1}{|hyp|} = \frac{2}{3} \approx \mathbf{0.667}$$

$$R = \frac{M_1}{|ref|} = \frac{2}{5} = \mathbf{0.400}$$

$$F_1 = \frac{2PR}{P + R} = \frac{2 \cdot \frac{2}{3} \cdot \frac{2}{5}}{\frac{2}{3} + \frac{2}{5}} = \frac{8/15}{16/15} = \mathbf{0.500}.$$

yaxshi referenceda yo'q, shuning uchun precision pasaydi.

referencedagi juda, qiziq, va tushib qoldi, shuning uchun recall ham



Intuitsiya

Kesishma = 2 ta unigram.

Faqat hypothesisda: yaxshi.

Faqat referenceda: juda, qiziq, va.

Kod ↔ formula: ROUGE-1

ROUGE-1 uchun soddalashtirilgan kod:

```
from collections import Counter

def rouge1(ref, hyp):
    r = ref.split()
    h = hyp.split()
    if not r or not h:
        return 0.0, 0.0, 0.0
    cr = Counter(r)
    ch = Counter(h)
    m1 = sum((ch & cr).values())
    p = m1 / len(h)
    rec = m1 / len(r)
    f1 = 2 * p * rec / (p + rec) if p + rec else 0.0
    return p, rec, f1

ref = "nlp juda qiziq va foydali"
hyp = "nlp juda foydali"
print(rouge1(ref, hyp)) # (1.0, 0.6, 0.75)
```

Kod → formula

Kod	Formula
ch & cr	clipped overlap
m1	M_1
m1 / len(h)	$P = \frac{M_1}{ hyp }$
m1 / len(r)	$R = \frac{M_1}{ ref }$
2*p*rec/(p+rec)	$F_1 = \frac{2PR}{P+R}$

Asosiy g'oya

Counter(h) & Counter(r) har bir unigram uchun eng kichik takror sonini oladi:

$$M_1 = \sum_w \min(c_{hyp}(w), c_{ref}(w)).$$

Bu takroriy so'z bilan bahoni sun'iy oshirishni cheklaydi.

ROUGE tuzog'i: yuqori ROUGE har doim yaxshi xulosa emas

Cheklovlar

- ROUGE asosan **leksik/n-gram ustma-ustlikni** o'lchaydi; ravonlik va grammatikani bevosita baholamaydi.
- Sinonim yoki parafraza ishlatilsa, ma'no to'g'ri bo'lsa ham ROUGE past chiqishi mumkin.
- Yuqori ROUGE faktlar to'g'ri degani emas: model manbada yo'q faktni qo'shib yuborishi mumkin.
- Faqat recallga qaralsa, uzun xulosa ko'proq birlikni qamrab, sun'iy yuqori ball olishi mumkin.

Amaliy qaror

ROUGE natijasini odatda bitta son bilan emas, bir nechta ko'rsatkich bilan qaraymiz:

$$P, \quad R, \quad F_1.$$

F_1 precision va recallni muvozanatlaydi: uzun, ortiqcha xulosani ham, juda qisqa xulosani ham nazorat qiladi.

Eslab qoling

ROUGE — **avtomatik va taxminiy** baho.

Yakuniy bahoda uni inson bahosi, fakt sodiqligi tekshiruvi va ma'no-asosli metrikalar, masalan BERTScore, bilan to'ldirish kerak.

Prezentatsiya rejasi

- 1 Kirish: rekurrentlikdan butunlay voz kechamiz
- 2 Self-attention: har pozitsiya hammaga qaraydi
- 3 Multi-head attention va positional encoding
- 4 Transformer bloki va umumlashtirish
- 5 Umumlashtirishni ROUGE bilan baholash
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: positional encoding va nisbatan erkin so'z tartibi

1. O'zbek tilida tartib nisbatan erkin

Masalan:

“men kitobni o'qidim” ↔ “kitobni men o'qidim”

Ikkalasi ham grammatik jihatdan mumkin, chunki **-ni** kelishik qo'shimchasi obyektни bildiradi.

Lekin tartib o'zgarsa, **fokus** va **ta'kid** ham o'zgaradi: ikkinchi gapda *kitobni* oldinga chiqarilgan.

Savol

Agar self-attention tartib signalini alohida bilmasa, model bir xil tokenlarning turli tartibini qanday farqlaydi?

2. Positional encoding nima beradi?

Self-attentionning o'zi tokenlar tartibini kodlamaydi. Positional encoding esa har token vektoriga **qayerda turgani** haqidagi signalni qo'shadi:

$$\tilde{x}_{pos} = x_{pos} + PE_{pos}.$$

Shunda model tokenning **mazmuni** bilan birga uning **pozitsiyasini** ham hisobga oladi.

Xulosa

PE o'zbek tiliga qat'iy so'z tartibini majburlamaydi. U faqat tartib signalini qo'shadi.

Korpus yetarli bo'lsa, model kelishik qo'shimchalari, SOV/OSV kabi tartib variantlari va fokus farqlarini birgalikda o'rganadi.

Sintez: RNN/LSTM'dan Transformer'ga

Jihat	RNN / LSTM	Transformer
Hisoblash oqimi	Ketma-ket (n qadam)	Parallel (1 qadam)
Uzoq bog'lanish	$O(n)$ yo'l, sust	$O(1)$ yo'l, kuchli
So'z tartibi	Tabiiy (rekurrent holatlar)	Positional encoding
Murakkablik	$O(n)$	$O(n^2)$ (attention)

Asosiy g'oya

Transformer rekurrentlikni **self-attention** bilan almashtiradi: to'liq parallel hisoblash va uzoq bog'lanishlarga kuchli e'tibor. Asosiy narx — $O(n^2)$ attention va positional encoding zarurati.

Asosiy manba: Vaswani va boshqalar (2017)

Manba

Vaswani, A., Shazeer, N., Parmar, N., va boshqalar (2017). ***Attention Is All You Need.*** *Advances in Neural Information Processing Systems* 30.

Nima uchun muhim?

- Rekurrent va konvolyutsion qatlamlarsiz, attentionni asosiy mexanizm sifatida ishlatadigan arxitekturani taklif qildi.
- Self-attention tokenlar orasidagi bog'lanishni parallel hisoblashga imkon berdi; bu katta til modellariga yo'l ochdi.
- Keyingi ko'plab NLP modellarida Transformer bloklari asosiy qurilish elementi sifatida ishlatildi.

Muhokama savoli

Self-attentionning narxi odatda $O(n^2)$, bu yerda n — tokenlar soni. Agar hujjat 10,000 token bo'lsa, attention matritsasi taxminan 10^8 juftlikni hisoblaydi.

O'zbek tilida qo'shimchalar sabab bir so'z bir nechta subword tokeniga bo'linishi mumkin. Bu token sonini oshirsa, attention hisoblash qanchalik og'irlashadi?

Bugungi maqsadlar bajarildi

- 1 ✓ **Tushuntirdik:** Transformer nima uchun rekurrent hisobdan voz kechishini: parallel hisoblash va uzoq masofali bog'lanishlar.
- 2 ✓ **Hisobladik:** self-attention vaznlarini:

$$\alpha = \text{softmax}([2, 1, 3]) = [0.245, 0.090, 0.665].$$

- 3 ✓ **Tushuntirdik:** multi-head attention, positional encoding, residual ulanish va LayerNorm g'oyasini.
- 4 ✓ **Hisobladik:** ROUGE-1 misolida:

$$P = 1.0, \quad R = 0.6, \quad F_1 = 0.75.$$

Keyingi ma'ruza (L13): Transformer'dan BERT va T5 ga

Bugun Transformer blokining asosiy mexanizmlarini ko'rdik: self-attention, multi-head attention, positional encoding, residual ulanish, LayerNorm va FFN.

Lekin har bir vazifa uchun Transformer modelini noldan o'qitish juda qimmat.

- **BERT** — oldindan o'qitilgan **enkoder** Transformer.
- **T5** — oldindan o'qitilgan **enkoder–dekoder** Transformer.
- L13 da ularni kichik **belgilangan ma'lumot**da fine-tuning qilib, aniq NLP vazifasiga moslashni ko'ramiz.

Asosiy g'oya

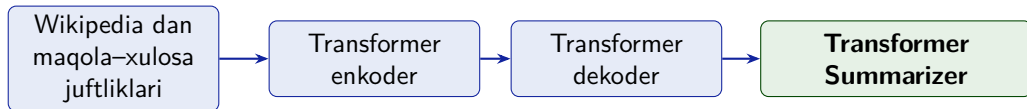
Transfer learning: model avval katta korpusdan umumiy til qonuniyatlarini o'rganadi, keyin ozroq ma'lumot bilan muayyan vazifaga moslashtiriladi.



L13 savoli

Qanday qilib bitta oldindan o'qitilgan model sentiment, tasniflash, savol-javob yoki umumlashtirish kabi turli vazifalarga moslashadi?

Keyingi Amaliyot: Transformer bilan matnni umumlashtirish



P12 da nimalar qilasiz?

- `TransformerSummarizer` modelini qurish
- encoder-decoder arxitekturasida self-attention va PE dan foydalanish
- maqola-xulosa juftliklarida o'qitish
- ROUGE-1 F_1 bilan baholash

Birinchi tekshiruv (assert)

L12 dagi qo'lda hisoblangan ROUGE-1 qiymati kodda ham chiqishi kerak:

```
assert abs(f1 - 0.750) < 1e-3
```

Ya'ni, siz yozgan kod qo'lda topilgan ROUGE-1 F_1 natijasi bilan mos kelishi kerak.

- ❶ Vaswani, A., Shazeer, N., Parmar, N., va boshqalar. (2017). *Attention Is All You Need*. *NeurIPS* 30.
- ❷ Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. *Text Summarization Branches Out*, ACL Workshop, 74–81.
- ❸ Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. *NAACL-HLT*, 4171–4186.
- ❹ Raffel, C., Shazeer, N., Roberts, A., va boshqalar. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. *JMLR*, 21.
- ❺ PyTorch documentation: `torch.nn.Transformer`; Google Research: rouge-score Python paketi.

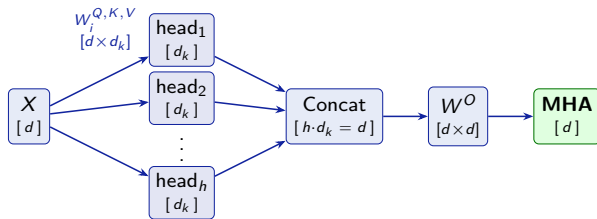
Ilova S1: multi-head attention to'liq matematikasi

h ta head, odatda har biri $d_k = d/h$ o'lchamli:

$$\text{head}_i = \text{softmax} \left(\frac{(XW_i^Q)(XW_i^K)^\top}{\sqrt{d_k}} \right) (XW_i^V)$$

bunda: $X \in \mathbb{R}^{n \times d}$; $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$
— har head proyeksiyalari. Softmax har so'rov tokeni uchun kalitlar bo'yicha olinadi. Natijalar konkatenatsiya qilinib, $W^O \in \mathbb{R}^{d \times d}$ bilan birlashtiriladi.

Har head turli bog'lanishlarni o'rganishi mumkin.
 $d_k = d/h$ tanlovi konkatenatsiyadan keyingi umumiy o'lchamni yana d ga teng saqlaydi.



O'lcham: $d \rightarrow d_k$ har head uchun; konkatenatsiya $hd_k = d$; W^O chiqishni yana d o'lchamga olib keladi.

Ilova S2: Post-LN va Pre-LN Transformer bloklari

Original Transformer: Post-LN

[$y = \text{Sublayer}(x)$, $z = \text{LayerNorm}(x+y)$.]

$$z = \text{LayerNorm}(x + \text{Sublayer}(x)).$$

Tartib:

Sublayer $\rightarrow$ Residual qo'shish $\rightarrow$ LayerNorm.

Bu ko'rinish original Transformer arxitekturasidagi klassik yozuvga mos keladi.

Zamonaviy amaliyotda ko'p uchraydigan: Pre-LN

[$y = \text{Sublayer}(\text{LayerNorm}(x))$, $z = x + y$.]

$$z = x + \text{Sublayer}(\text{LayerNorm}(x)).$$

Tartib:

LayerNorm $\rightarrow$ Sublayer $\rightarrow$ Residual qo'shish.

Chuqur Transformerlarni o'qitishda gradient oqimi barqarorroq bo'lishi uchun ko'p ishlatiladi.

Asosiy farq nimada?

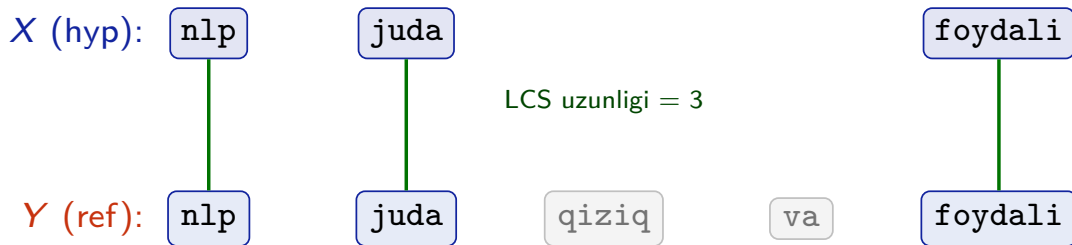
Ikkalasida ham asosiy g'oya bir xil: [sublayer natijasi + residual yo'l.] Farq faqat **LayerNorm qayerda qo'llanishida**: Post-LN normallashni qo'shishdan keyin qiladi, Pre-LN esa sublayerdan oldin qiladi. Asosiy dars slaydida Post-LN beriladi, chunki u original Transformer ta'rifiga mos.

Ilova S3: ROUGE-L (eng uzun umumiy ketma-ketlik)

ROUGE-L unigram emas, **LCS** (*longest common subsequence*) ga asoslanadi:

$$R_{\text{lcs}} = \frac{|\text{LCS}(X, Y)|}{|Y|}, \quad P_{\text{lcs}} = \frac{|\text{LCS}(X, Y)|}{|X|}, \quad F_{\text{lcs}} = \frac{2P_{\text{lcs}}R_{\text{lcs}}}{P_{\text{lcs}} + R_{\text{lcs}}}.$$

bunda: X — hypothesis, Y — reference; $|\text{LCS}(X, Y)|$ — tartibni saqlagan eng uzun umumiy qism-ketma-ketlik uzunligi. Tokenlar yonma-yon bo'lishi shart emas.



$X = \text{nlp juda foydali}$, $Y = \text{nlp juda qiziq va foydali} \Rightarrow \text{LCS} = \text{nlp juda foydali}$ (uzunligi 3; qiziq, va o'tkazib yuboriladi).